BRIEF.md

Context Decay and Intent Preservation in Creative and Technical Workflows

A White Paper
February 2026 | v1.0-draft

Gyles Gyesie
ORCID: 0009-0008-3394-9243
github.com/GGyesieSP/BRIEF.md
DOI: 10.5281/zenodo.18614773
hello@gylesgyesie.com

This document accompanies the BRIEF.md Specification (v1.0-draft).
Specification: CC-BY 4.0 | Feedback welcome

Contents

Executive Summary
1. The Problem: Context Decay
1.1 A Proposed Model of Differential Decay
1.2 The Reconstruction Problem
1.3 The Effort Asymmetry
1.4 The Unnamed Problem
2. AI-Assisted Workflows and the Context Illusion
2.1 The Illusion of Shared Understanding
2.2 Three Proposed Mechanisms
3. Existing Approaches and Their Limitations
3.1 Project Identity Files
3.2 Architectural Decision Records
3.3 Domain-Specific Metadata Standards
3.4 Centralised Documentation Platforms
3.5 AI Memory Features
3.6 Independent Convergence
3.7 The Gap
4. The BRIEF.md Approach
4.1 Intellectual Foundations
4.2 Core Structure
4.3 Key Design Decisions
4.4 What BRIEF.md Does Not Do
5. Novel Contributions
6. Open Research Questions
7. Practical Applications
8. Adoption Considerations
9. Relationship to Existing Standards and Conventions
10. Risks and Misuse Patterns
11. Potential Implications for AI Training Data
12. Beyond Continuity: Post-Release Discoverability
13. Conclusion
Appendix A: Complete BRIEF.md Example (Music)
Appendix B: Complete BRIEF.md Example (Specification)
Appendix C: Glossary
Appendix D: Additional Design Considerations
References

Executive Summary

Modern creative and technical work increasingly spans long time horizons, multiple tools, frequent interruptions, and AI-assisted workflows. While contemporary software excels at preserving the state of a project (files, parameters, versions), it largely ignores the preservation of project-level intent: the purpose, constraints, decisions, and rationale that shape work over time.

This white paper introduces and examines a phenomenon we term context decay: the systematic loss of project intent over time and across tools. It proposes a model of five distinct context types that decay at different rates, analyses how AI-assisted workflows may accelerate this decay, and presents BRIEF.md: a lightweight, open, human-readable context file designed to make project intent explicit and portable.

BRIEF.md is not a project management system, a comprehensive documentation platform, or an AI tool. It is a minimal text file that captures what a project is, what it is not, why it exists, which key decisions have been made and why, and what questions remain open. It synthesises established practices (architectural decision records from software engineering, project identity files like README.md, and domain-specific metadata conventions) into a single, hierarchically composable context file that applies across creative and technical domains.

This paper provides the conceptual foundation for the BRIEF.md specification. It is intended for practitioners, researchers, and tool builders interested in the problem of context decay, and is written to stand independently of the specification and implementation guide.

A note on terminology: throughout this paper, “context file” or “context artifact” refers to a file whose purpose is to make project intent explicit, such as BRIEF.md. When referring to the outputs of creative or technical work (code, recordings, designs), we use the terms “project outputs” or “work.”

1. The Problem: Context Decay

Knowledge workers commonly describe significant re-entry costs when returning to projects after interruptions. Research on task switching (Monsell, 2003), cognitive load (Sweller, 1988), and workplace interruptions (Mark, Gudith & Klocke, 2008; Czerwinski, Horvitz & Wilhite, 2004) has documented productivity losses, increased stress, and degraded performance during context switching. Yet tooling interventions have focused overwhelmingly on task management rather than intent reconstruction.

The term context decay, as used in this paper, refers to the systematic loss of project-level intent (purpose, constraints, decision rationale, and relational awareness) over time and across tools. It is distinct from, though related to, established concepts such as task switching costs (Monsell, 2003) and cognitive load (Sweller, 1988). Context decay is not simply the time required to resume work; it is the degradation of the understanding that informed the work in the first place.

This is an observable pattern in professional practice, though we note that formal empirical measurement of its rate and effects remains an open research question. The model presented here is a proposed framework intended to sharpen discussion and guide future study, not a claim of established scientific fact.

1.1 A Proposed Model of Differential Decay

What is commonly referred to as “context” in professional workflows appears to consist of at least five distinct types of knowledge, each with different properties and, plausibly, different decay characteristics:

Identity context (what this is): the most stable layer. Practitioners rarely forget what a project is, though scope may drift gradually without explicit anchoring.

Motivational context (why this exists): arguably the most fragile high-value context. After an interruption of days or weeks, practitioners commonly describe remembering what a project is but not why it felt important. The emotional and strategic charge that drove decisions tends to fade.

Constraint context (what is excluded and why): constraints during active work often exist as tacit intuitions (“that doesn’t feel right”), consistent with Polanyi’s (1966) observation that we can know more than we can tell. After a break, the intuition may be gone even though the constraint still matters.

Decision context (why this choice over alternatives): the decision itself often persists because it is embedded in the work itself (the code, the mix, the design). But the rationale behind the decision tends to fade. This can create hesitancy: a practitioner sees a choice they no longer understand and is reluctant to change it because they do not know what it was protecting.

Relational context (how this connects to the larger picture): awareness of how a component fits a larger system, how a song fits an album, or how a feature reflects a product strategy. This awareness tends to fade as the number of concurrent projects increases.

Context Type	What It Answers	Proposed Decay Rate	Example
Identity	What is this?	Slow (drifts under pressure)	This is a folk song about restlessness
Motivational	Why does this exist?	Medium (first to blur)	To capture the feeling of wanting to leave
Constraint	What is excluded?	Fast (often tacit)	No full-band arrangement
Decision	Why this choice?	Fast to medium	Chorus doesn't resolve; resolution would betray the core
Relational	How does this connect?	Medium (fades with complexity)	Song's sparseness reflects artist's overall aesthetic

Note: The decay rates described above are based on practitioner observation and are presented as a proposed framework. Formal empirical validation of differential decay rates across these five types is identified as an open research question in Section 6.

1.2 The Reconstruction Problem

When practitioners reconstruct context from project outputs alone (reading code, listening to mixes, reviewing designs), they rebuild an approximation of their original understanding. This approximation has predictable characteristics:

- It is incomplete: practitioners recover what is visible in the work but not the reasoning behind it.
- It is biased toward current salience: reconstruction reflects present mood and priorities, not the original ones.
- It lacks awareness of negative space: what was deliberately excluded and why tends to be lost entirely.

If this model is accurate, it implies that every interruption introduces not random drift, but directed drift toward whatever is salient at the moment of resumption. This is a hypothesis worth testing: that interruptions do not merely cost time but may systematically alter creative and technical direction.

1.3 The Effort Asymmetry

The differential decay model has a practical implication that any intervention must address: the context types that decay fastest and matter most are also the hardest to capture.

Identity context ("what this is") is the easiest to write and the least urgently needed; practitioners rarely lose it. Constraint context ("what this is not") and decision context ("why this choice") are the most valuable to preserve but the most difficult to articulate, because they require documenting tacit knowledge at the moment when a practitioner is most engaged in the work itself. Stopping to write down why you chose something feels like interrupting flow to do paperwork.

This creates a structural tension: the sections of any context file that address the fastest-decaying, highest-value context will be the ones most likely to be left empty. Any practical approach to context preservation must acknowledge this asymmetry rather than treating all sections as equally approachable.

1.4 The Unnamed Problem

Perhaps the most significant barrier to addressing context decay is that most practitioners do not recognise it as a named, solvable problem. The friction of returning to a paused project is experienced as inevitable, "just how it is." Practitioners tend to blame themselves for not remembering rather than observing that their tools preserve state but not intent.

This matters because unnamed problems do not attract solutions. Documentation standards, project management tools, and version control systems have all been built to address problems that were identified and named: change tracking, task coordination, collaboration at scale. Context decay (the systematic loss of the reasoning behind the work) has not been framed as a distinct problem with distinct solutions. The status quo of accepting the cost is, in practice, the primary competitor to any intervention in this space.

2. AI-Assisted Workflows and the Context Illusion

A contemporary factor that warrants specific examination is the role of AI assistants in creative and technical workflows. The following analysis describes observed patterns in AI-assisted work. While these patterns appear consistent across informal practitioner accounts and the authors' direct observation, they have not yet been the subject of controlled empirical study.

2.1 The Illusion of Shared Understanding

When a practitioner describes a project to a current-generation AI assistant, the assistant responds with apparent understanding: mirroring language, asking relevant-seeming questions, generating output that feels aligned. This creates a subjective experience of collaboration.

Some AI platforms now offer project-level features (Claude's Projects allow uploading reference files, and ChatGPT supports custom instructions), but these provide static context injection rather than a dynamic, evolving understanding of a project's identity, constraints, and decision history. The sense of deep shared context is, in functional terms, still largely an illusion: comprehension has been externalised to a system whose "understanding" is bounded by what the user explicitly provides at session start.

This dynamic is a specific instance of cognitive offloading: the use of external tools to reduce internal mental load (Risko & Gilbert, 2016). Research on AI-assisted work has documented how over-reliance on AI tools can reduce practitioners' own cognitive engagement, with downstream effects on critical thinking and independent reasoning (Gerlich, 2025; Guingrich, Mehta & Bhatt, 2026). In the context of project work, the concern is not simply that AI systems lack context (this is widely understood), but that the convincing performance of context may reduce a practitioner's motivation to maintain their own explicit records. If this effect occurs, net project understanding in the human-AI system could decrease when AI enters the workflow.

2.2 Three Proposed Mechanisms

We propose three mechanisms by which AI-assisted workflows may accelerate context decay. These are presented as hypotheses warranting investigation, not as established findings.

The outsourcing dynamic. This mechanism is closely related to cognitive offloading as described in the education and cognitive science literature (Risko & Gilbert, 2016). When working alone, practitioners maintain context because no one else holds it. When working with human collaborators, context is distributed but durable. AI creates a third mode: context feels distributed but is only as durable as what the user explicitly provides. Even with project-level features (uploaded files, custom instructions), practitioners may reduce their own context maintenance because the AI appears to be tracking the project, yet the AI has no independent model of how the project has evolved across sessions.

The velocity dynamic. AI dramatically increases output velocity. A practitioner who might explore two options in an hour can now explore twenty. In pre-AI work, the relative slowness of execution created natural checkpoints for reflection, time to consider why a specific choice was being made. Increased velocity may reduce these built-in opportunities for intent reflection.

The local coherence dynamic. AI-generated output tends to be locally coherent: each individual piece reads well, sounds right, or looks professional. But local coherence can mask

global incoherence. Without explicit constraints and decision rationale, there may be no mechanism for either the practitioner or the AI to detect when output contradicts choices made in earlier sessions.

3. Existing Approaches and Their Limitations

Several existing conventions and tools address aspects of the intent preservation problem. Understanding their strengths and limitations clarifies the gap that BRIEF.md is designed to address.

3.1 Project Identity Files (README.md, agent.md, AGENTS.md)

README.md is the most widely adopted project root file. It effectively establishes project identity, is portable (file-based), and is human-readable. However, READMEs are typically oriented toward current state rather than decision history. They rarely capture decision rationale, constraints with reasoning, or the evolution of project intent over time.

Agent files (.cursorrules, CLAUDE.md, AGENTS.md) serve a related but distinct function: providing instructions to AI coding assistants. AGENTS.md, an open standard for giving AI coding agents project context, has seen rapid adoption, with industry reports estimating 40,000–60,000 open-source implementations by late 2025 (OpenAI, 2025; Linux Foundation, 2025), and is supported by multiple major AI tools. These files demonstrate strong and growing demand for project-root context files. However, they are AI-specific (written as instructions for agents, not as context for humans), instruction-focused (they tell AI how to behave, not why the project exists), and do not include decision records, constraint documentation, or cross-domain applicability. Notably, Gemini CLI now supports reading context files from multiple directory levels and merging them (Google, 2025), but this capability is applied only to AI agent instructions.

3.2 Architectural Decision Records (ADRs)

Proposed by Michael Nygard in an influential 2011 blog post (Nygard, 2011), ADRs are short documents recording architecture decisions with context, rationale, status, and consequences. They represent perhaps the strongest existing practice for capturing decision context. However, ADRs are stored as separate files (one per decision), creating friction; they assume the reader already knows project identity and purpose; and they have not been adopted outside software engineering. The core insight of ADRs (that decisions should be documented with rationale and rejected alternatives) is sound and forms one of the intellectual foundations of BRIEF.md.

3.3 Domain-Specific Metadata Standards

Standards such as MusicBrainz (musicbrainz.org), Dublin Core (dublincore.org), and Schema.org standardise vocabularies within domains and enable interoperability. They capture descriptive metadata effectively but do not address intent, rationale, decision history, or creative constraints. They are structured data formats optimised for machines rather than human-readable prose.

3.4 Centralised Documentation Platforms

Platforms such as Notion, Confluence, and Google Docs provide rich documentation capabilities but separate documentation from the project itself. This spatial separation tends to create a maintenance gap that widens over time. Documentation platforms also create vendor dependency for context that, arguably, should travel with the project.

3.5 AI Memory Features

Major AI providers are building persistent memory features into their products. These take different approaches to context persistence. ChatGPT's memory remains proprietary and vendor-locked, with no dedicated export feature. Claude introduced memory import/export in October 2025, allowing users to export and transfer memories between services (Anthropic, 2025). However, these features still represent platform-specific implementations rather than open, tool-agnostic formats, and none provides the structured, human-editable project context that a dedicated context file offers.

3.6 Independent Convergence

Anecdotal evidence suggests that individual practitioners have independently adopted similar patterns: placing a markdown file at the project root that captures context, decisions, and intent for AI-assisted workflows, and updating it after each session. Examples include personal productivity workflows shared in developer communities, such as Donn Felker's "llm-context.md" approach (Felker, 2025). These informal, uncoordinated approaches (typically personal productivity patterns rather than published specifications) suggest that the underlying need is real and currently unaddressed by any standard.

3.7 The Gap

No widely adopted representation of project intent currently exists that integrates decision records with project identity, composes hierarchically, applies across creative and technical domains, is human-readable and human-editable, and is not tied to a specific vendor or tool.

A further observation: all existing project-root context files are designed AI-first: instructions for agents that happen to be human-readable. BRIEF.md inverts this, as discussed in Section 4.3.

4. The BRIEF.md Approach

BRIEF.md is a simple Markdown file placed at the root of a project directory. It captures project intent in a human-readable, machine-parseable format designed to survive pauses, tool changes, collaboration, and time. The full specification is published separately; this section describes the design rationale.

4.1 Intellectual Foundations

BRIEF.md synthesises three established practices:

- **Project identity files** (README.md, agent.md): the convention of placing a descriptive file at the project root.
- **Architectural decision records** : the practice of documenting decisions with rationale and rejected alternatives.
- **Domain-specific metadata ontologies** : standardised vocabularies for describing projects within specific domains.

The synthesis combines these into a single context file with properties that none possess individually: hierarchical composability, cross-domain applicability, integrated decision records alongside project identity, explicit documentation of negative space, and tracking of intentional ambiguity.

4.2 Core Structure

Every BRIEF.md contains structured metadata (project name, type, creation date) followed by up to five core sections, each mapping to a distinct type of context:

Section	Context Type Addressed	Purpose
What This Is	Identity	Clear, concise project description
What This Is NOT	Constraint	Explicit boundaries and exclusions with reasoning
Why This Exists	Motivational	Purpose, motivation, and intended outcome
Key Decisions	Decision	Choices made with rationale and rejected alternatives
Open Questions	Relational / All	Unresolved questions and intentional ambiguities

Optional domain-specific extensions (e.g., for music production, software architecture, visual storytelling) can supplement these core sections with specialised vocabularies. Extensions are designed to be community-defined and are not required.

4.3 Key Design Decisions

Human-readable first. BRIEF.md uses Markdown prose, not JSON, YAML, or database schemas. The file should be useful even if no tool ever parses it. Tools are encouraged to be lenient in parsing and to accept human variation in formatting.

Hierarchical context through folder nesting. BRIEF.md files placed in nested directories form an implicit context hierarchy. A tool opening a project can walk up the directory tree and accumulate context from parent BRIEF.md files. A song-level BRIEF.md inherits awareness of its album and artist context with zero configuration. This hierarchical model does not enforce inheritance; parent context is advisory, not prescriptive. Children make their own decisions.

Decision records embedded in the project file. Rather than storing decisions as separate files (the ADR pattern), BRIEF.md integrates decision records alongside project identity, constraints, and open questions. This reduces friction and provides unified context. Both minimal (one-sentence rationale) and full (with alternatives considered) formats are supported.

Explicit negative space. The “What This Is NOT” section documents what a project is deliberately excluding and why. Constraints during active work are often tacit; they exist as intuitions that evaporate after a break. Making them explicit is intended to prevent the most common re-entry error: changing something that seemed fine but contradicts an original constraint.

Intentional ambiguity tracking. The “To Keep Open” subsection within Open Questions names things that should remain unresolved. In creative work, unresolved tensions are often features rather than defects: a narrative ambiguity, an aesthetic tension, an open interpretation. To our knowledge, no widely adopted project documentation standard includes a dedicated section for questions that should deliberately remain unresolved.

Cross-domain portability. The core structure is domain-agnostic. The same five sections work for a song, a software feature, a research paper, or a film scene. Domain-specific depth is provided through optional extensions rather than baked into the core format.

Colocation with the project. BRIEF.md lives in the project directory. It travels with the project: when you open the project, the context is there; when you archive, share, or move the project, the context comes with it. This colocation (placing intent alongside the work itself rather than in a separate system) is a deliberate design choice intended to eliminate the spatial separation that causes documentation platforms to drift from the projects they describe.

Human-first, AI-readable. Existing project-root context files are designed AI-first: instructions for agents that happen to be human-readable. BRIEF.md inverts this. It is written for humans (in prose, with rationale, emphasising meaning) and structured so that AI tools can also parse and

use it. This inversion reflects the position that the primary audience for project intent is the practitioner, not the machine.

Solo and team applicability. The format is identical for solo creators and teams, but the dynamics differ. For solo practitioners, BRIEF.md is primarily communication with a future self: the sections most likely to fade (decisions, constraints, motivation) are the ones to prioritise. For teams, BRIEF.md serves as alignment infrastructure: preventing re-litigation of resolved debates, making uncertainty visible, and enabling onboarding without archaeological digging through old messages and documents.

4.4 What BRIEF.md Does Not Do

It is important to be precise about the scope of the context file:

- BRIEF.md does not replace project management systems, task trackers, or planning documents.
- It does not enforce workflows, methodologies, or creative processes.
- It does not make AI systems understand a project; it provides a structured reference point that both humans and AI tools can read.
- It does not guarantee that project intent will be preserved; it provides a mechanism for explicit capture that is otherwise absent.
- It does not require tooling, automation, or AI integration to be valuable. The file is useful the moment it has a project name and a sentence of description.

5. Novel Contributions

BRIEF.md builds on established practices. The following elements represent, to our knowledge, contributions that do not exist in prior published work. We present these as design contributions and conceptual framings rather than as empirically validated claims.

5.1 Hierarchical Context Composition

While folder-based context files exist in AI agent configuration (CLAUDE.md, AGENTS.md, and Gemini CLI all support hierarchical context discovery as of 2025), these implementations are instruction-focused and software-specific. BRIEF.md extends this pattern to project intent preservation across creative and technical domains, with advisory rather than prescriptive inheritance. ADRs are flat. READMEs do not compose. The BRIEF.md design (in which a tool walks up the directory tree and accumulates context from parent files) provides a way for a project component to know its place within a larger structure without explicit configuration. A

song knows it is part of an album that is part of an artist's body of work. A software feature knows it is part of a product that reflects a company's engineering philosophy.

5.2 Decision Records for Creative Domains

ADRs have existed in software engineering since 2011 (Nygard, 2011) but have not, to our knowledge, been adapted to creative work. In creative domains, "decisions" include aesthetic choices, emotional goals, and intentional ambiguities, types of decisions not typically associated with the architectural and technical choices ADRs were designed to capture. The adaptation of decision records to creative work, particularly the "To Keep Open" concept (where unresolved questions are features rather than bugs), addresses a gap in existing documentation practices. While "Last Responsible Moment" decision-deferral and assumption tracking are established practices in agile methodologies, to our knowledge no existing documentation standard includes a dedicated section for items consciously chosen to remain unresolved as a creative asset.

5.3 Negative Space as a Formal Section

While 'Non-Goals' sections appear in software design documentation (notably design document templates used at companies such as Google, and various RFC formats), these typically address scope boundaries rather than creative constraints with rationale. BRIEF.md adapts this practice to creative work and frames it as essential context preservation rather than optional scope management. This section addresses constraint context, which, under the proposed decay model, is among the fastest to decay and most consequential to lose.

5.4 Universal Core with Domain Extensions

The architecture of a domain-agnostic core (identity, purpose, decisions, questions) extended by community-defined domain-specific vocabularies is, to our knowledge, a compositional design not present in current project documentation standards. The approach draws on the extensibility principles of semantic web ontologies (where a shared core vocabulary can be supplemented by domain-specific additions) but applies them to human-readable project files rather than machine-readable structured data. This architecture is proposed but not yet tested at scale; its viability depends on community adoption and governance structures that remain to be established.

5.5 Context Decay as a Design Framing

The framing of intent preservation as a first-class design concern for tools (rather than as a human memory problem or a documentation discipline problem) is, as far as we are aware, a distinct contribution. The underlying phenomenon has been studied under related terms: task switching costs in cognitive science (Monsell, 2003; Rogers & Monsell, 1995), cognitive load in instructional design (Sweller, 1988; Sweller, van Merriënboer & Paas, 1998), workplace interruption costs in HCI (Mark, Gudith & Klocke, 2008; Mark, Gonzalez & Harris, 2005;

Czerwinski, Horvitz & Wilhite, 2004), cognitive offloading onto external tools and AI systems (Risko & Gilbert, 2016; Guingrich, Mehta & Bhatt, 2026), and architectural knowledge vaporisation in software engineering. The term “context decay” as applied to project-level intent preservation, and the proposed five-type taxonomy of differential decay, represent a novel framing of these established concerns, one that shifts the locus of responsibility from human memory to tool design.

6. Open Research Questions

This paper proposes a model and a context file specification. Neither has been subjected to controlled empirical study. The following research questions represent what we consider the most productive avenues for validation, refinement, or refutation.

6.1 Empirical Validation of Context Decay

- Do the five proposed context types decay at measurably different rates? If so, which types are most critical to preserve?
- Do interruptions systematically alter creative or technical direction, or is drift random?
- What is the relationship between interruption duration and extent of context loss?

6.2 Effects of Explicit Context Preservation

- Does a structured context file (such as BRIEF.md) measurably reduce re-entry time after interruptions?
- Does explicit context preservation affect project completion rates?
- Does externalising tacit creative intent change the nature of the creative work itself, for better or worse?
- What is the interaction between context preservation and productive creative ambiguity? Does documenting constraints reduce or enhance creative exploration?

6.3 AI-Context Interaction

- Does AI output quality change measurably when an explicit project context file is available versus when it is not?
- Does the availability of structured context reduce the three proposed AI-context traps (outsourcing, velocity, local coherence)?
- How do practitioners’ context maintenance behaviours change when working with AI assistants versus working alone?

6.4 Adoption and Maintenance

- What are the actual friction points in creating and maintaining context files?
- Which sections do practitioners find most and least valuable?
- Are effects consistent across hobbyist and professional settings?
- Does tool-assisted context maintenance (where AI proposes updates that humans review) affect adoption and quality compared to manual maintenance?

7. Practical Applications

While empirical validation is pending, the following applications illustrate how BRIEF.md could function in practice across different domains.

7.1 Music Production

Music production may be a particularly instructive domain for studying context decay and testing context preservation approaches, for several reasons. Music has a natural hierarchy (artist, album, song) that maps directly to the folder-nesting model with zero configuration. It depends heavily on aesthetic consistency across components; a song that sounds excellent in isolation but does not fit the album is a context failure. Music production has high and visible abandonment rates: producers frequently describe abandoning sessions when re-entry friction exceeds motivation, making context decay's costs tangible and immediate. The domain is also experiencing rapid AI integration, making all three proposed AI-context dynamics (outsourcing, velocity, local coherence) observable in daily practice.

Critically, decision records are genuinely novel in this domain. ADRs exist in software engineering; nothing equivalent exists in music production, where decisions (aesthetic choices, emotional goals, production constraints) are overwhelmingly tacit. A producer opening a song project months after creating it could read a BRIEF.md capturing the emotional intent, the aesthetic constraints (e.g., no programmed drums, room reverb only), the decision to keep the guitar slightly out of tune (with the rationale that perfection destroys the intimate quality), and the intentional ambiguity in the lyric. A DAW integration could read this file and use it to frame AI-generated suggestions in emotional terms, warn about changes that contradict documented constraints, and provide context without the producer needing to reconstruct it from memory.

7.2 Software Development

Developers already maintain context files for AI tools (agent.md, .cursorrules). BRIEF.md extends this pattern by adding decision records, hierarchical composition, and human-readable intent alongside AI instructions. A feature-level BRIEF.md could record why a particular

architectural approach was chosen over alternatives, what the feature deliberately does not do, and open questions about future direction, all discoverable by a new team member or an AI coding assistant.

7.3 Film and Visual Media

A film production could use nested BRIEF.md files at the studio, franchise, film, and scene levels. Thematic intent, visual constraints, narrative decisions, and intentional ambiguities could be captured once and referenced throughout production, particularly valuable during post-production, which often occurs months after principal photography when original creative intent has faded.

7.4 Research and Academic Work

Research projects that span months or years, involve multiple collaborators, and evolve through many iterations are natural candidates for explicit intent documentation. A BRIEF.md could capture the research question, methodological decisions with rationale, what the study deliberately does not attempt, and open questions under active investigation.

8. Adoption Considerations

Any context preservation approach must contend with fundamental friction: the effort of documentation falls at the moment of deepest engagement, the payoff is delayed, and there is no immediate feedback loop. BRIEF.md is designed with these tensions in mind but does not eliminate them.

8.1 Design for Low Friction

The specification supports a minimal viable entry point: a project name, type, creation date, and a single sentence of description. Empty sections are acceptable. A half-finished BRIEF.md is explicitly positioned as a starting point, not a failure. Both minimal (one-sentence) and full (with alternatives) decision formats are supported. The goal is to lower the threshold for initial creation while allowing depth for those who want it.

8.2 Tool-Assisted Maintenance

The most promising path to sustainable adoption likely involves tool integration: generating BRIEF.md skeletons at project creation, prompting for context at natural decision points, and using AI to propose updates after working sessions that humans then review. Review is lower friction than authoring from scratch. However, the specification is designed to be useful without any tool support.

8.3 Openness and Portability

BRIEF.md is released under CC-BY 4.0. It is tool-agnostic, vendor-independent, and designed to be extended by communities. Context captured in BRIEF.md is not locked into any platform or service. This openness is a deliberate design choice reflecting the position that a creator's understanding of their own project should be an asset they own and control.

9. Relationship to Existing Standards and Conventions

BRIEF.md does not aim to replace existing conventions. It occupies a specific position within the ecosystem of project documentation:

Convention	Strength	Gap BRIEF.md Addresses
README.md	Project identity, setup	Decision rationale, constraints, intent evolution
agent.md / AGENTS.md	AI tool instructions, growing adoption	Human readability, decision history, hierarchy, non-software domains
ADRs	Decision rationale with alternatives	Unified file, project identity, creative domains, hierarchy
Domain metadata (MusicBrainz, Dublin Core)	Standardised vocabularies	Intent, rationale, human-readable prose, constraints
AI memory features	Convenience, automatic	Portability, inspectability, user ownership, vendor independence
Notion / Confluence	Rich documentation	Colocation with project, portability, survival across tools

BRIEF.md is designed to coexist with these conventions. A project could have a README.md (setup and usage), an agent.md (AI tool instructions), and a BRIEF.md (project intent and decisions), each serving its specific purpose. While complementary in function, BRIEF.md specifically addresses a gap where intent preservation has historically been no convention's job.

10. Risks and Misuse Patterns

Every lightweight specification that gains adoption attracts overuse. Documenting foreseeable failure modes early is essential to maintaining the file's utility over time. The following patterns represent the most significant risks.

10.1 Bureaucratisation

BRIEF.md could become long, formal, and mandatory within organisations. Contributors might feel pressure to fill every section. Updates could lag behind reality. The file becomes performative rather than useful, written for compliance, not clarity. When this happens, practitioners stop reading the file, and context decay resumes with the added cost of maintaining a document nobody trusts. As a general guide: if a BRIEF.md takes more than five minutes to read, consider whether it is doing too many jobs. Complex projects with many documented decisions may legitimately be longer, but the file should remain focused on context rather than becoming a comprehensive reference document.

10.2 Treating the File as Canonical Truth

If BRIEF.md is treated as authoritative rather than advisory, it can actively harm creative and exploratory work. Some of the best decisions in any project come from productive deviation: discovering something better than what was originally planned. If documented constraints become enforced constraints, the file prevents the very evolution it was designed to support. BRIEF.md documents understanding, not correctness. The “What This Is NOT” section does not prevent crossing boundaries; it ensures that the crossing is conscious and deliberate. When a documented constraint no longer serves the project, updating the file is itself a decision worth recording.

10.3 AI Prompt Accumulation

As AI tools become central to workflows, there is a risk that BRIEF.md becomes a large system prompt, filled with tool-specific instructions, prompt engineering patterns, and machine-readable directives rather than human-readable context. This would undermine the file’s core purpose. The distinction matters: “the aesthetic is sparse and intimate” is context; “always respond in JSON format with max 500 tokens” is an instruction. BRIEF.md should remain useful even if no AI tool ever reads it. Tool-specific data belongs in designated tool sections, clearly separated from the human-readable core.

10.4 The Meta-Guardrail

The simplest heuristic for whether BRIEF.md is being used well: is the file useful? If practitioners return to the project and the file helps them resume work, it is working. If practitioners skip the file because it is too long, too stale, or too formal, it is being misused. As a general guide, the file should remain readable in under five minutes, editable by hand, and valuable even without tool support.

11. Potential Implications for AI Training Data

The following section is deliberately speculative and forward-looking. It describes a potential downstream consequence of BRIEF.md adoption, not a motivation for it. The primary value of BRIEF.md is to the practitioner; the implications described here are secondary.

11.1 The Process Gap in AI Training Data

AI models that generate music, writing, design, and code are trained almost exclusively on outputs: finished songs, published text, shipped software, final designs. They learn what creative work looks like. They do not learn how it was made, why it was made that way, or what was deliberately rejected along the way.

The result is AI tools that are fluent in creative outputs but have no model of creative process: the decisions, the trade-offs, the constraints, and the moments where something was considered and rejected. This gap limits AI's ability to function as a process-aware collaborator rather than a pattern-matching output generator.

11.2 What BRIEF.md Data Would Contain

If BRIEF.md were adopted at scale, it would produce a category of data that does not currently exist in structured, portable form: records of human creative decision-making. Unstructured versions of this data exist (artist interviews, director commentaries, design critiques), but not as machine-parseable records linked to specific projects. This structured data would include intent alongside output (not just "here is a folk song" but "this song exists to capture the feeling of being stuck in a small town"), decisions with rationale and rejected alternatives, constraints with reasoning, explicit negative space (what was deliberately excluded), records of intentional ambiguity (what the creator chose not to resolve), and hierarchical aesthetic context (how individual choices relate to larger creative frameworks).

Such data could, in principle, enable AI systems to develop models of aesthetic constraint (understanding that "sparse and intimate" implies a set of things that should not be present), to reason about creative trade-offs (understanding that choosing one quality means sacrificing another), and to maintain coherence across creative hierarchies (understanding how a song-level choice relates to an album-level aesthetic). These are capabilities that are difficult or impossible to reliably reverse-engineer from finished outputs alone.

11.3 Important Caveats

These implications are contingent on adoption at meaningful scale, and any use of BRIEF.md data for AI training would raise significant questions of consent, ownership, and data quality. BRIEF.md content is deeply personal; it encodes how someone thinks creatively. Any training use would need to be explicitly opted into, clearly attributed, and under the creator's control. The

CC-BY 4.0 licence on the specification does not imply any licence on the content of individual BRIEF.md files.

Furthermore, the data would only have value if it is honest. If creators write BRIEF.md entries knowing they will be used for AI training, the entries may be shaped by that knowledge, potentially less candid, less specific, and therefore less useful. The most valuable data would come from files written purely for the practitioner's own use and shared voluntarily after the fact.

12. Beyond Continuity: Post-Release Discoverability

During development, BRIEF.md is a private continuity tool. But once a project ships, parts of that context become potential discovery infrastructure. The intent, constraints, and aesthetic framing that guided creation are precisely the metadata that helps audiences, collaborators, licensors, and internal teams find and contextualise the work later.

12.1 The Lifecycle Shift

Today, the rich context that informed a project's creation is typically lost entirely (it lived in someone's head), trapped in internal tools (Slack threads, Notion pages, email chains), or reduced to crude tags that flatten the work's identity. BRIEF.md creates a potential pathway from rich creative context during production to rich discoverable metadata after release.

Not everything in a BRIEF.md should be public. The split is intuitive: aesthetic direction, genre, themes, influences, and production approach could aid discovery; Key Decisions, Open Questions, and "What This Is NOT" typically reveal internal process and should remain private. The creator decides; the default is private.

12.2 Illustrative Applications

Music sync licensing. A music supervisor searching for "late-night restlessness, observational, sparse acoustic" currently mostly relies on genre, lyrics and mood tags, often reviewing hundreds of results. Creator-authored intent metadata (describing what a song is about at the thematic level, not just what it sounds like) could enable search by meaning rather than by surface-level attributes.

Internal component discovery. Software organisations frequently duplicate engineering effort because existing components cannot be found. BRIEF.md metadata describing what a component does, what it is designed for, and what it deliberately does not do could make internal search more effective by surfacing intent alongside implementation details.

Content cataloguing. Streaming platforms curating thematic collections ("films about people who feel trapped but find quiet acceptance") currently rely on manual editorial review or

algorithmic inference. Creator-authored thematic intent is richer and more trustworthy than either.

12.3 Creator-Authored vs Algorithmic Metadata

Algorithmic tagging (AI analysing a song and guessing its mood, or reading code and inferring its purpose) is scalable but has systematic blind spots. It struggles to detect intent (a deliberately out-of-tune guitar may be tagged “low quality” rather than “intentional imperfection”). It cannot detect negative space (the absence of drums might be an oversight or a deliberate choice). It may not reliably detect thematic depth (whether a song about a train is literally about transportation or metaphorically about escape). Creator-authored metadata from BRIEF.md can capture these dimensions, though only to the extent that the creator documents them. The marginal cost of making selected parts discoverable after release is low; the metadata was already created for the practitioner’s own continuity.

12.4 The Adoption Feedback Loop

Post-release discoverability addresses one of context preservation’s fundamental adoption challenges: the delayed and invisible payoff. If a BRIEF.md maintained for continuity also improves how work is found and understood after release, the file produces two returns: one for the practitioner’s future self and one for the project’s public life. This second, more tangible payoff could reinforce the maintenance habit that the first, delayed payoff struggles to sustain alone.

13. Conclusion

The problem this paper examines (the systematic loss of project intent over time and across tools) is supported by established research on task switching costs, cognitive load, and workplace interruptions, and by the independent emergence of informal context preservation practices among practitioners. Yet it remains under-theorised as a distinct phenomenon and under-addressed by current tooling. AI-assisted workflows appear to intensify this problem by creating the illusion of shared context while providing only limited, session-bounded understanding, by increasing output velocity beyond the pace of reflective intent maintenance, and by producing locally coherent output that may mask global incoherence.

BRIEF.md is a proposed response: a minimal, open, human-readable context file for making project intent explicit and portable. Its design draws on established practices (architectural decision records, project identity files, domain metadata) and contributes several elements that, to our knowledge, have not been combined in existing project documentation standards: hierarchical context composition applied to intent preservation, the adaptation of decision

records to creative domains, negative space as essential context preservation, intentional ambiguity tracking, and the framing of context decay as a design concern rather than solely a cognitive one. These contributions and their relationship to existing prior art are discussed in detail in Section 5.

We present this work with appropriate epistemic humility. The context decay model is a proposed framework, not an established finding. The differential decay rates are hypothesised, not measured. The effectiveness of BRIEF.md as an intervention is untested in controlled settings. We have identified specific research questions (Section 6) that would support, refine, or refute the claims made here.

What we can state with confidence is that the problem is real (practitioners across domains report the experience of losing intent) and that no widely adopted, open solution currently addresses it. BRIEF.md is offered as a starting point: a concrete proposal around which discussion, critique, research, and practical experimentation can converge.

The specification, implementation guide, and supporting materials are publicly available and released under CC-BY 4.0. Feedback, critique, and collaboration are invited.

Appendix A: Complete BRIEF.md Example (Music)

The following is a complete example of a BRIEF.md file for a music project, demonstrating the core sections and one domain extension.

```
**Project:** Echo Valley
**Type:** song
**Extensions:** sonic_arts, narrative_creative
**Status:** development
**Created:** 2026-01-15
**Updated:** 2026-02-01

# What This Is
A folk song about small-town restlessness told through
late-night train imagery. Sparse acoustic arrangement with
fingerpicked guitar and ambient field recordings.

# What This Is NOT
- Not a love song (the restlessness is about place, not a person)
- Not a full-band arrangement (the sparseness is the point)
- Not a protest song (observation, not anger)

# Why This Exists
To capture the specific feeling of lying awake in a small town
hearing a train pass through, wanting to be on it, knowing
you won't be.

# Key Decisions

### WHAT: Verse 2 stays in the same location as Verse 1
**WHY:** The song is about stasis. Keeping the narrator in bed
makes "I'll go tomorrow" land as self-deception.
**WHEN:** 2026-01-20
**ALTERNATIVES CONSIDERED:**
- Train station in V2 → rejected: false momentum
- Porch (compromise) → rejected: still implies action

# Open Questions

## To Resolve
- [ ] Keep raw train recording or replace with clean sample?

## To Keep Open
- Whether narrator addresses themselves or someone who left
  (both readings add something)

# SONIC ARTS

## Aesthetic Direction
Sparse, intimate, analog warmth.

## Production Constraints
- No programmed drums
- Room reverb only
- Maximum 8 tracks
```

Appendix B: Complete BRIEF.md Example (Specification)

The following is a complete example of a BRIEF.md file for this specification project itself, demonstrating the core sections applied to a non-creative, technical domain. No domain extension is used, illustrating that the core sections work independently.

```
**Project:** BRIEF.md Specification
**Type:** open_standard
**Status:** development
**Created:** 2026-01-15
**Updated:** 2026-02-10

# What This Is
An open specification for a lightweight, human-readable context file
that preserves project intent across time, tools, and collaborators.
Defines a universal core structure (identity, boundaries, motivation,
decisions, open questions) with optional domain-specific extensions.

# What This Is NOT
- Not a project management system (no tasks, timelines, or owners)
- Not a comprehensive documentation platform (no wikis, no knowledge base)
- Not an AI tool or AI memory replacement (no intelligence, no inference)
- Not a database schema or knowledge graph (optimises human recall,
  not machine queries)
- Not a methodology or workflow prescription (records outputs of
  decisions, not how to make them)
- Not a replacement for README.md or agent.md (occupies a distinct,
  complementary space)

# Why This Exists
Tools preserve state but not intent. When work pauses and resumes,
practitioners lose awareness of why decisions were made, what
constraints mattered, and what was deliberately excluded. This cost
is invisible, unnamed, and accepted as inevitable. BRIEF.md exists
to make project intent explicit, portable, and persistent, so that
re-entry becomes continuation rather than reconstruction.

# Key Decisions

### WHAT: Markdown prose, not JSON or YAML
**WHY:** The file must be useful even if no tool ever parses it.
Human readability is the primary design constraint. Structured data
formats optimise for machines; BRIEF.md optimises for a person
returning to a project after weeks away.
**WHEN:** 2026-01-15
**ALTERNATIVES CONSIDERED:**
- JSON: rejected, unreadable by humans without tooling
- YAML: rejected, better than JSON but still feels like
  configuration, not prose
- Custom format: rejected, unnecessary friction, no ecosystem

### WHAT: Context hierarchy through folder nesting, not explicit linking
**WHY:** Relationships between projects should be self-evident from
structure, not maintained through configuration. A song in an album
folder inherits album context without anyone declaring the relationship.
Zero configuration means zero maintenance burden for hierarchy.
**WHEN:** 2026-01-15
**ALTERNATIVES CONSIDERED:**
- Explicit links between files: rejected, creates maintenance burden,
  links break when files move
- Configuration file declaring relationships: rejected, adds complexity,
```

```
  requires tooling to manage
- Flat structure (no hierarchy): rejected, loses relational context
  entirely

### WHAT: Decision records embedded in the project file, not separate
  documents
**WHY:** ADRs store each decision as a separate file in a decisions
directory. This is architecturally clean but creates friction: you
navigate away from context to write one, and there is no unified view
of the project. Embedding decisions alongside identity, constraints,
and open questions provides unified context in a single file.
**WHEN:** 2026-01-15
**ALTERNATIVES CONSIDERED:**
- Separate ADR files per decision: rejected, proven pattern but
  friction of file creation limits adoption, especially outside
  software
- Decisions in external documentation: rejected, spatial separation
  causes drift

### WHAT: "What This Is NOT" as a formal core section
**WHY:** Constraint context is among the fastest to decay and most
consequential to lose. Constraints during active work are often tacit
intuitions that vanish after a break. Making them explicit prevents
the most common re-entry error: changing something that seemed fine
but contradicts an original constraint. While "Non-Goals" sections
appear in some software design documentation, framing negative space
as essential context preservation (not optional scope management) is
a deliberate design choice.
**WHEN:** 2026-01-15

### WHAT: "To Keep Open" subsection for intentional ambiguity
**WHY:** In creative work, unresolved tensions are often features
rather than defects. No widely adopted project documentation standard
includes a dedicated section for questions that should deliberately
remain unresolved. This section gives practitioners permission to name
what should stay open, rather than treating all open questions as
problems to be solved.
**WHEN:** 2026-01-20

### WHAT: Domain-agnostic core with optional community-defined
  extensions
**WHY:** The same five core sections work for a song, a software
feature, a research paper, or a film scene. Domain-specific depth
is provided through extensions rather than baked into the core,
keeping the base format simple while allowing communities to add
vocabularies relevant to their domain.
**WHEN:** 2026-01-15
**ALTERNATIVES CONSIDERED:**
- Domain-specific formats from the start: rejected, fragments the
  standard, limits cross-domain portability
- No extension mechanism: rejected, core alone is too shallow for
  specialised work
- Single monolithic format covering all domains: rejected, becomes
  bloated and intimidating

### WHAT: Human-first, AI-readable (not AI-first)
**WHY:** Existing project-root context files (agent.md, .cursorrules,
AGENTS.md) are written as instructions for AI agents that happen to
be human-readable. BRIEF.md inverts this: written for humans in prose
with rationale, structured so AI tools can also parse it. The primary
audience for project intent is the practitioner, not the machine.
**WHEN:** 2026-01-20
**ALTERNATIVES CONSIDERED:**
```

```
- AI-first with human readability as secondary: rejected, optimises
  for the wrong audience and risks becoming brittle prompt engineering

### WHAT: Parent context is advisory, not prescriptive
**WHY:** Creative and exploratory work requires deviation. If parent
BRIEF.md files enforced constraints on children, the hierarchy would
prevent the very evolution it aims to support. Advisory context
informs without constraining. Tools may warn about conflicts but do
not enforce.
**WHEN:** 2026-01-20
**ALTERNATIVES CONSIDERED:**
- Prescriptive inheritance: rejected, prevents productive deviation,
  makes the hierarchy a constraint enforcer
- No parent awareness at all: rejected, loses the value of relational
  context

### WHAT: Colocation with the project directory, not external storage
**WHY:** Spatial separation between documentation and work is a
primary cause of documentation drift. When context lives in the same
directory as the work, it travels with the project: archiving,
sharing, and moving the project automatically preserves context.
**WHEN:** 2026-01-15
**ALTERNATIVES CONSIDERED:**
- Centralised documentation platform: rejected, creates spatial
  separation, vendor dependency
- Database or API-based storage: rejected, requires tooling, not
  human-editable

# Open Questions

## To Resolve
- [ ] Should the specification recommend a maximum file length or
      remain silent on size?
      Context: Complex projects with many decisions will naturally
      produce longer files. A hard limit risks being unrealistic;
      no guidance risks bureaucratic bloat. Current approach uses a
      readability heuristic (five minutes) rather than a line count.
- [ ] How should conflict detection between parent and child BRIEF.md
      files work?
      Context: The spec currently says "implementation-specific."
      As adoption grows, clearer guidance may be needed on what
      constitutes a conflict and how tools should surface it.
- [ ] Is the proposed extension architecture viable at scale?
      Context: No extensions have been formally defined. No
      community governance exists. The composability of a universal
      core with domain-specific vocabularies is architecturally sound
      in principle but depends on community adoption.
- [ ] Does BRIEF.md measurably reduce re-entry time after
      interruptions?
      Context: The claim that explicit context preservation helps
      is intuitively strong but empirically untested. Controlled
      studies are needed to validate, quantify, or refute the effect.
- [ ] Does AI output quality change when a BRIEF.md is available?
      Context: Anecdotal evidence suggests AI tools perform better
      with structured context, but no controlled comparison exists.
- [ ] Does externalising tacit creative intent change the nature of
      the creative work itself?
      Context: There is a genuine risk that documenting constraints
      and decisions could reduce productive ambiguity or creative
      freedom. This interaction needs investigation.

## To Keep Open
- Whether BRIEF.md adoption ultimately depends on tool integration
```

```
  or can sustain on manual use alone (both paths have merit and the
  answer may differ across domains and user types)
- Whether the "context decay" framing is best understood as a design
  problem, a cognitive problem, or both (the paper argues for design,
  but this is a position, not a settled finding)
- The boundary between context and instruction: where does project
  intent end and tool configuration begin? (This tension is productive
  and may not have a clean resolution)
```

Appendix C: Glossary

Context decay. The systematic loss of project-level intent (purpose, constraints, decision rationale, and relational awareness) over time and across tools. Proposed in this paper as a framework; empirical validation pending.

Context illusion. The subjective experience of shared understanding with an AI system whose project awareness is limited to what the user explicitly provides. A specific instance of cognitive offloading (Risko & Gilbert, 2016) applied to project intent: practitioners reduce their own context maintenance because the AI appears to be tracking the project. Proposed as a phenomenon warranting investigation.

Differential decay. The proposed model in which different types of context (identity, motivational, constraint, decision, relational) fade at different rates after an interruption.

Negative space. What a project is deliberately not: explicit boundaries and exclusions documented with reasoning.

Intentional ambiguity. Questions or tensions within a project that a creator has decided should remain unresolved because the ambiguity itself serves the work.

Hierarchical context composition. The BRIEF.md design pattern in which folder nesting implies context nesting, and tools walk up the directory tree to accumulate context from parent files.

Domain extension. An optional, community-defined vocabulary that adds domain-specific depth to a BRIEF.md file (e.g., SONIC ARTS for music production, SYSTEM DESIGN for software architecture).

Appendix D: Additional Design Considerations

The following topics provide additional context on design decisions and potential applications. They are included for completeness and for readers interested in the finer details of the specification.

Walking Down the Hierarchy

The core context-discovery mechanism is walking up: when a tool opens a file, it discovers the bigger picture by reading parent BRIEF.md files. But tools can also walk down the hierarchy, scanning child directories to show what a collection contains. Use cases include an artist-level view that surfaces themes across all albums and songs, a product-level view that identifies which features have open questions, and pattern detection across a body of work ("three of our last five songs have unresolved tension as a core theme; is this a signature or a rut?"). Walking down is an intentional user action rather than something that triggers automatically on every file open.

Tool Integration Levels

Tool builders interested in supporting BRIEF.md can approach integration incrementally. Level 1 (read and display) involves parsing metadata and displaying core sections. Level 2 (hierarchy-aware) adds walking up the directory tree to discover and display accumulated parent context. Level 3 (write support) generates BRIEF.md skeletons on project creation and prompts for context at natural decision points. Level 4 (AI integration) feeds BRIEF.md content to AI tools at session start and proposes updates after working sessions. Each level adds value; none requires the others.

External Ontologies and Vocabularies

Domain extensions may recommend external ontologies for standardised vocabularies, for example MusicBrainz genre tags for a SONIC ARTS extension or standard architecture pattern catalogues for a SYSTEM DESIGN extension. These are recommendations, not requirements: free text always works. When multiple projects follow the same ontologies, tools can offer smarter autocomplete, validation, and cross-project analysis. But the specification prioritises human readability over machine-optimised vocabularies.

Present-Tense Utility

The primary value proposition of BRIEF.md is re-entry: the file helps when you return to a project after time. But the file is more likely to be maintained if it is also useful during active work. Potential present-tense applications include AI tools that read BRIEF.md at session start and behave differently as a result, search across one's own projects ("which of my songs have unresolved tension as a theme?"), collaboration onboarding ("read the BRIEF.md before we start"), and creative prompting ("your open questions for this project are..."). The more BRIEF.md does today, the more likely practitioners are to maintain it for tomorrow.

Continuity over Optimisation

BRIEF.md prioritises continuity over optimisation and clarity over completeness. It is not designed to produce the most thorough documentation possible, but to preserve the minimum context required for meaningful resumption. A BRIEF.md with only “What This Is” and one Key Decision is infinitely more valuable than no BRIEF.md at all. The specification explicitly positions incompleteness as acceptable because the biggest adoption barrier for documentation standards is perfectionism: practitioners who feel they cannot do it “properly” often do not start at all.

Proposed Research Methodology

For researchers interested in empirically testing the claims in this paper, a four-phase study design has been outlined in accompanying research materials. Phase 1 establishes baseline measurements by having participants resume paused projects without structured context files, measuring re-entry time, decision drift, and context reconstruction accuracy. Phase 2 introduces BRIEF.md as an intervention and repeats the measurements after a comparable dormancy period. Phase 3 examines AI interaction specifically, comparing AI output quality and practitioner satisfaction with and without BRIEF.md available to the AI tool. Phase 4 combines quantitative metrics with qualitative interviews on perceived coherence, confidence, and creative flow. Music production is recommended as the primary study domain due to its clear hierarchical structure, measurable abandonment rates, and the prominence of tacit aesthetic decision-making.

References

AGENTS.md. (2025). AGENTS.md: An open standard for AI coding agents. GitHub repository. https://github.com/agentsmd/agents.md

Anthropic. (2025). Importing and exporting your memory from Claude. Claude Help Center. https://support.claude.com/en/articles/12123587-importing-and-exporting-your-memory-from-claude

Czerwinski, M., Horvitz, E., & Wilhite, S. (2004). A diary study of task switching and interruptions. Proceedings of the SIGCHI Conference on Human Factors in Computing Systems (CHI '04), 175–182. https://doi.org/10.1145/985692.985715

Dublin Core Metadata Initiative. (n.d.). Dublin Core Metadata Element Set. https://dublincore.org

Felker, D. (2025, March 24). Productive LLM coding with an llm-context.md file. https://www.donnfelker.com/productive-llm-coding-with-an-llm-context-md-file/

Gerlich, M. (2025). AI tools in society: Impacts on cognitive offloading and the future of critical thinking. Societies, 15(1), 6. https://doi.org/10.3390/soc15010006

Google. (2025). Gemini CLI: Context files documentation. https://github.com/google-gemini/gemini-cli

Gyesie, G. (2026). Context decay and intent preservation in creative and technical workflows: The BRIEF.md specification (v1.0-draft). Zenodo. https://doi.org/10.5281/zenodo.18614773

Guingrich, R. E., Mehta, D., & Bhatt, U. (2026). Belief offloading in human-AI interaction. arXiv preprint arXiv:2602.08754. https://arxiv.org/abs/2602.08754

Linux Foundation. (2025, December 9). Linux Foundation announces the formation of the Agentic AI Foundation (AAIF). https://www.linuxfoundation.org/press/linux-foundation-announces-the-formation-of-the-agentic-ai-foundation

Mark, G., Gudith, D., & Klocke, U. (2008). The cost of interrupted work: More speed and stress. Proceedings of the SIGCHI Conference on Human Factors in Computing Systems (CHI '08), 107–110. https://doi.org/10.1145/1357054.1357072

Mark, G., Gonzalez, V. M., & Harris, J. (2005). No task left behind? Examining the nature of fragmented work. Proceedings of the SIGCHI Conference on Human Factors in Computing Systems (CHI '05), 321–330. https://doi.org/10.1145/1054972.1055017

Monsell, S. (2003). Task switching. Trends in Cognitive Sciences, 7(3), 134–140. https://doi.org/10.1016/S1364-6613(03)00028-7

MusicBrainz. (n.d.). MusicBrainz: Open music encyclopaedia. https://musicbrainz.org

Nygard, M. (2011, November 15). Documenting architecture decisions. Cognitect Blog. https://cognitect.com/blog/2011/11/15/documenting-architecture-decisions

OpenAI. (2025, December 9). OpenAI co-founds the Agentic AI Foundation under the Linux Foundation. https://openai.com/index/agentic-ai-foundation/

Polanyi, M. (1966). The tacit dimension. University of Chicago Press. https://press.uchicago.edu/ucp/books/book/chicago/T/bo6035368.html

Risko, E. F., & Gilbert, S. J. (2016). Cognitive offloading. Trends in Cognitive Sciences, 20(9), 676–688. https://doi.org/10.1016/j.tics.2016.07.002

Schema.org. (n.d.). Schema.org. https://schema.org

Rogers, R. D., & Monsell, S. (1995). Costs of a predictable switch between simple cognitive tasks. Journal of Experimental Psychology: General, 124(2), 207–231. https://doi.org/10.1037/0096-3445.124.2.207

Sweller, J. (1988). Cognitive load during problem solving: Effects on learning. Cognitive Science, 12(2), 257–285. https://doi.org/10.1207/s15516709cog1202_4

Sweller, J., van Merriënboer, J. J. G., & Paas, F. G. W. C. (1998). Cognitive architecture and instructional design. Educational Psychology Review, 10(3), 251–296. https://doi.org/10.1023/A:1022193728205